

SISTEMA DE GESTIÓN ACADÉMICA

Documentación Técnica de Backend

Microservicio 2 — Inscripciones

Endpoints · Modelo de datos · Reglas de negocio · Arquitectura

Versión 1.0
Julio 2026

Índice

Índice	2
1. Introducción	4
1.1 Objetivo del documento	4
1.2 Alcance del microservicio	4
1.3 Contexto general del sistema	4
2. Arquitectura del backend	5
2.1 Diagrama de arquitectura	5
2.2 Patrón de capas	5
2.3 Estructura de carpetas	6
2.4 Módulos / recursos expuestos	6
3. Especificaciones técnicas de desarrollo	7
3.1 Stack tecnológico	7
3.2 Configuración de la aplicación (config/config.py)	7
3.3 Variables de entorno (.env)	7
Ejemplo de archivo .env	8
3.4 Inicialización de la aplicación	8
3.5 Despliegue	8
4. Seguridad y autenticación	10
4.1 AuthCommon	10
4.2 Sistema de permisos granular	10
4.3 JWT	11
4.4 Redis	11
5. Formato estándar de respuesta y códigos HTTP	12
5.1 Envelope de respuesta	12
Respuesta exitosa	12
Respuesta de error	12
5.2 Códigos HTTP utilizados	12
5.3 Manejo de errores	12
6. Modelo de datos	14
6.1 Diagrama entidad-relación	14
6.2 Referencias externas (a otros microservicios)	14
6.3 Detalle de entidades	14
Inscripcion	14
Clase	15
Asistencia	15
Evaluacion	15

Calificacion.....	16
ResultadoAcademico.....	17
ResultadoPlan.....	17
Certificado	17
6.4 Catálogos y valores iniciales (seed)	18
7. Comunicación entre microservicios.....	19
7.1 Cliente hacia el Microservicio 1 (Planes)	19
Endpoints consumidos del Microservicio 1	19
Datos consumidos de las reglas académicas del plan.....	19
7.2 Comunicación hacia el Microservicio Auth	19
8. Catálogo de endpoints	21
8.1 Inscripciones — /inscripciones	21
8.2 Clases — /clases	23
8.3 Asistencias — /asistencias	25
8.4 Evaluaciones — /evaluaciones	26
8.5 Calificaciones — /calificaciones	29
8.6 Resultados académicos — /resultados-academicos	30
8.7 Resultados de plan — /resultados-planes.....	32
8.8 Certificados — /certificados	34
9. Reglas de negocio transversales	36
10. Limitaciones conocidas y puntos pendientes.....	36

1. Introducción

1.1 Objetivo del documento

Este documento describe técnicamente el backend del Microservicio 2 — Inscripciones, componente del Sistema de Gestión Académica. Su propósito es servir como referencia para el equipo de desarrollo y para equipos consumidores (frontend u otros microservicios), detallando la arquitectura, los modelos de datos, las reglas de negocio, los endpoints disponibles y las especificaciones necesarias para instalar, configurar y mantener el servicio.

1.2 Alcance del microservicio

El Microservicio 2 administra el ciclo académico posterior a la inscripción: gestión de clases, registro de asistencia, evaluaciones y calificaciones, cálculo de resultados académicos por comisión, seguimiento del avance del alumno dentro de su plan de estudio, y emisión de certificados.

Concretamente, expone recursos para:

- Inscripciones de alumnos a comisiones-asignatura.
- Clases dictadas dentro de una comisión-asignatura.
- Registro de asistencia por clase.
- Evaluaciones (parciales, recuperatorios, finales, trabajos prácticos) y sus calificaciones.
- Resultados académicos por inscripción (aprobado / regular / desaprobado / libre).
- Resultados de avance del plan de estudio del alumno.
- Emisión de certificados de aprobación o participación.

1.3 Contexto general del sistema

El sistema está compuesto por tres microservicios independientes que se comunican entre sí vía HTTP/REST, detrás de un proxy nginx que sirve los distintos frontends:

- Microservicio Auth — autenticación, usuarios y catálogo de permisos (librería compartida AuthCommon).
- Microservicio 1 — Planes: legajos, comisiones-asignatura, planes de estudio y reglas académicas (presentismo, notas de aprobación/regularización).
- Microservicio 2 — Inscripciones (este proyecto): objeto de esta documentación.

El Microservicio 2 no almacena copia local de legajos, comisiones ni planes de estudio: los consulta en tiempo real contra el Microservicio 1 a través de un cliente HTTP interno (clients/planes_cliente.py).

2. Arquitectura del backend

2.1 Diagrama de arquitectura

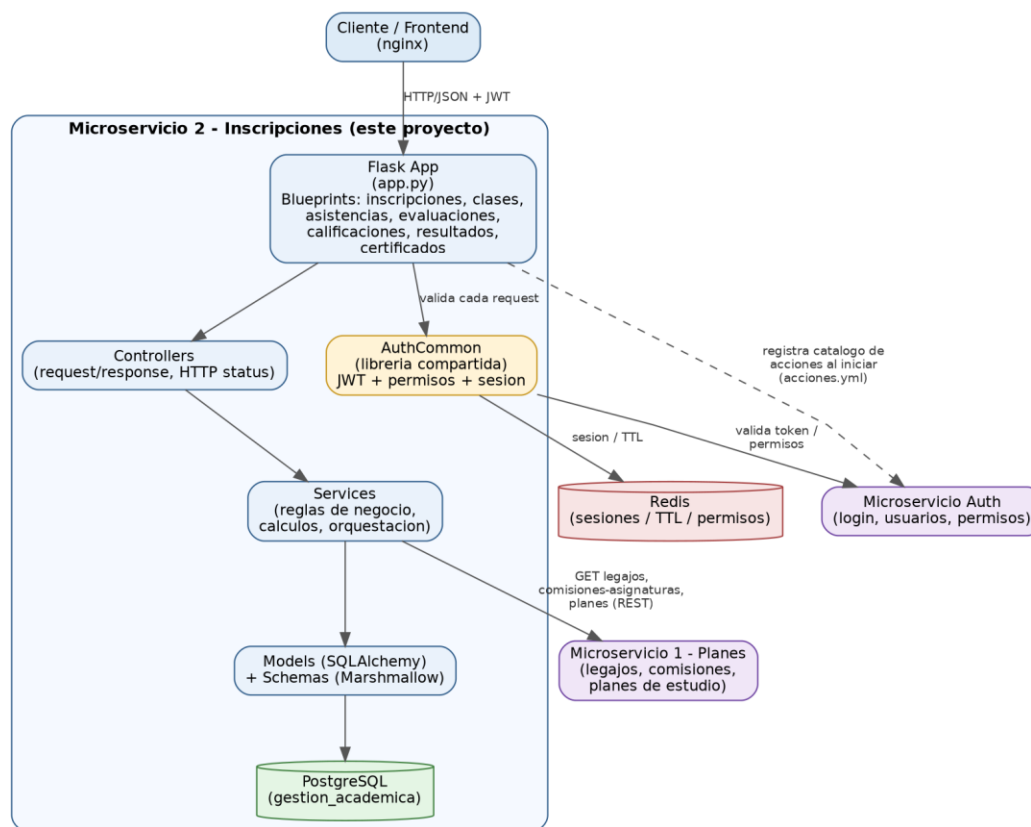


Figura 1. Arquitectura general y comunicación entre microservicios.

2.2 Patrón de capas

El backend sigue una arquitectura en capas clásica, con separación estricta de responsabilidades. El flujo de una petición HTTP atraviesa las siguientes capas, en orden:

- **Routes (routes/*.py)**: define el path, el método HTTP y el permiso requerido para cada acción, usando Blueprints de Flask.
- **AuthCommon (librería compartida)**: intercepta cada request, valida el JWT, la sesión en Redis y el permiso asociado antes de continuar.
- **Controllers (controllers/*.py)**: reciben el request, validan el JSON de entrada con el schema de Marshmallow correspondiente, invocan al service, serializan la respuesta y traducen las excepciones a códigos HTTP.
- **Services (services/*.py)**: contienen toda la lógica de negocio — validaciones cruzadas, cálculos, orquestación entre entidades y comunicación con otros microservicios.
- **Models (models/*.py)**: definen las tablas mediante SQLAlchemy ORM.
- **Schemas (schemas/*.py)**: definen la serialización (salida) y validación (entrada) de datos mediante Marshmallow.

Ningún controller accede directamente a los modelos: toda interacción con la base de datos pasa por la capa de services, lo que centraliza las reglas de negocio y facilita su prueba y mantenimiento.

2.3 Estructura de carpetas

```
backend/
├── app.py                # Punto de entrada, registro de blueprints
├── extensions.py         # Instancias globales (db, ma)
├── exceptions.py        # BusinessException (excepción de negocio)
├── config/config.py      # Configuración (BD, JWT, Redis, AuthCommon)
├── clients/             # Clientes HTTP a otros microservicios
│   └── planes_cliente.py # Cliente al Microservicio 1 (Planes)
├── controllers/         # Un controller por recurso
├── routes/              # Un blueprint por recurso
├── services/            # Lógica de negocio, un service por recurso
├── models/              # Modelos SQLAlchemy
├── schemas/             # Schemas Marshmallow
├── seed/               # Carga inicial de catálogos
├── utils/
│   ├── response.py      # success_response / error_response
│   ├── logger.py        # Configuración de logging
│   └── auth_registro.py  # Registra acciones.yml contra Auth al iniciar
├── acciones.yml         # Catálogo de permisos del microservicio
├── requirements.txt
├── Dockerfile
└── .env
```

2.4 Módulos / recursos expuestos

El microservicio registra 8 blueprints, cada uno bajo su propio prefijo de ruta:

Campo	Detalle
/inscripciones	Alta, consulta, modificación y baja de inscripciones.
/clases	ABM de clases dentro de una comisión-asignatura.
/asistencias	Registro y consulta de asistencia por clase.
/evaluaciones	ABM de evaluaciones (parciales, finales, recuperatorios, TPs).
/calificaciones	Registro y consulta de calificaciones por evaluación.
/resultados-academicos	Generación y consulta de resultados académicos por comisión.
/resultados-planes	Consulta y administración del avance del plan de estudio del alumno.
/certificados	Emisión, consulta y administración de certificados.

3. Especificaciones técnicas de desarrollo

3.1 Stack tecnológico

Campo	Detalle
Lenguaje	Python 3
Framework web	Flask 3.1
ORM	Flask-SQLAlchemy 3.1 / SQLAlchemy 2.0
Serialización / validación	Flask-Marshmallow 1.5 / Marshmallow 4.3
Autenticación	Flask-JWT-Extended + librería interna AuthCommon
Base de datos	PostgreSQL (driver psycopg 3, binario) — SQLite como fallback local
Sesiones / permisos	Redis
Cliente HTTP saliente	requests
CORS	Flask-Cors
Configuración por entorno	python-dotenv
Testing	pytest
Contenedores	Docker (Dockerfile propio) + docker-compose

3.2 Configuración de la aplicación (config/config.py)

La configuración se centraliza en la clase Config, que carga variables desde un archivo .env ubicado en la raíz de backend/. Puntos clave:

- El motor de base de datos es seleccionable vía DB_ENGINE ("postgres" o cualquier otro valor, que cae en SQLite local — pensado para desarrollo sin Docker).
- JWT_SECRET_KEY es obligatoria: si no está definida, la aplicación falla al iniciar (RuntimeError).
- Los tokens JWT se leen únicamente desde headers (JWT_TOKEN_LOCATION = ["headers"]).
- Expiración de tokens: access token 15 minutos, refresh token 7 días.
- El endpoint /status está exceptuado de autenticación (AUTH_COMMON_ENDPOINTS_EXCEPTUADOS).

3.3 Variables de entorno (.env)

Campo	Tipo	Nulo	Clave	Descripción
DB_ENGINE	postgres sqlite	No	—	Motor de base de datos a utilizar.
SQLITE_PATH	string	Sí	—	Ruta del archivo SQLite (solo si DB_ENGINE≠postgres).
POSTGRES_HOST	string	No*	—	Host del contenedor/servidor PostgreSQL.
POSTGRES_PORT	int	No*	—	Puerto de PostgreSQL (5432 por defecto).

Campo	Tipo	Nulo	Clave	Descripción
POSTGRES_DB	string	No*	—	Nombre de la base de datos (gestion_academica).
POSTGRES_USER	string	No*	—	Usuario de conexión.
POSTGRES_PASSWORD	string	No*	—	Contraseña de conexión.
AUTH_SERVICE_URL	URL	No	—	Base URL del microservicio Auth (registro de acciones).
JWT_SECRET_KEY	string	No	—	Clave de firma de JWT. Obligatoria, sin default.
AUTH_COMMON_REDIS_URL	URL	No	—	Conexión a Redis para sesiones y permisos.
AUTH_COMMON_SESSION_TTL	int (seg)	No	—	Tiempo de vida de la sesión (900s = 15 min).
PLANES_SERVICE_URL	URL	No	—	Base URL del Microservicio 1 (Planes).

* Solo obligatorias si DB_ENGINE=postgres.

Ejemplo de archivo .env

```
DB_ENGINE=postgres
SQLITE_PATH=db.sqlite

POSTGRES_HOST=postgres-inscripcion
POSTGRES_PORT=5432
POSTGRES_DB=gestion_academica
POSTGRES_USER=postgres
POSTGRES_PASSWORD=postgres

AUTH_SERVICE_URL=http://backend:5000
JWT_SECRET_KEY=<clave-secreta>

AUTH_COMMON_REDIS_URL=redis://redis:6379/0
AUTH_COMMON_SESSION_TTL=900

PLANES_SERVICE_URL=http://planes-backend:5000
```

Nota: los nombres de host (backend, redis, planes-backend) corresponden a los nombres de servicio dentro de la red de Docker Compose. Para ejecutar el microservicio fuera de Docker, deben reemplazarse por localhost (con el puerto correspondiente de cada servicio).

3.4 Inicialización de la aplicación

Al ejecutar app.py directamente (modo desarrollo), la aplicación realiza tres pasos dentro del contexto de la app:

- db.create_all() — crea las tablas en base a los modelos definidos, si no existen.
- cargar_datos_iniciales() — puebla las tablas de catálogo (estados y tipos) solo si están vacías.
- registrar_acciones() — envía el contenido de acciones.yml al Microservicio Auth (POST /auth/acciones) para sincronizar el catálogo de permisos.

El servidor corre en 0.0.0.0:5000 con debug=True.

3.5 Despliegue

El proyecto se despliega mediante Docker. A nivel del sistema completo, existe un `docker-compose.yml` general y archivos específicos por microservicio (`docker-compose.auth.yml`, `docker-compose.inscripcion.yml`, `docker-compose.planes.yml`), además de un `nginx` como reverse proxy que sirve los distintos frontends (`auth`, `inscripciones`, `login`, `planes`) y enruta el tráfico a cada backend.

Según las variables de entorno relevadas, la infraestructura del Microservicio 2 incluye, como mínimo, los siguientes contenedores/servicios:

- `backend` — la aplicación Flask de este microservicio (puerto interno 5000).
- `postgres-inscripcion` — base de datos PostgreSQL dedicada a este microservicio.
- `redis` — compartido para sesiones y permisos (`AuthCommon`).

4. Seguridad y autenticación

4.1 AuthCommon

AuthCommon es una librería interna compartida entre los tres microservicios del sistema (Auth, Inscripciones y Planes). Se inicializa en `app.py` (`AuthCommon(app)`) e intercepta las peticiones entrantes para validar autenticación y autorización de forma centralizada, antes de que la petición llegue al controller.

Sus responsabilidades principales son:

- Validar el token JWT recibido en el header de la petición.
- Verificar la sesión activa del usuario en Redis (con expiración configurable por `AUTH_COMMON_SESSION_TTL`).
- Verificar que el usuario autenticado posea el permiso asociado al endpoint solicitado (decorador `requires_permission`).
- Exponer el usuario autenticado a la capa de negocio a través de `g.id_usuario`, usado en todos los services para registrar auditoría (`id_usuario_creacion` / `id_usuario_modificacion`).

El endpoint `/status` es la única ruta exceptuada de este proceso, útil como health-check de infraestructura.

4.2 Sistema de permisos granular

Cada endpoint declara explícitamente el permiso que requiere, mediante el decorador `requires_permission` de AuthCommon, con el formato `inscripcion.<recurso>.<acción>`:

```
asistencia_bp.route("/", methods=["POST"])(
    requires_permission("inscripcion.asistencias.crear")(agregar_asistencias)
)
```

Al iniciar la aplicación, `utils/auth_registro.py` lee el catálogo completo de permisos definido en `acciones.yml` y lo registra contra el Microservicio Auth (`POST /auth/acciones`), de modo que ese catálogo quede disponible para ser asignado a roles de usuario desde el módulo de administración.

Catálogo de acciones (permisos) del microservicio, por recurso:

Campo	Detalle
inscripciones	crear · leer · actualizar · eliminar
clases	crear · leer · actualizar · eliminar
asistencias	crear · leer · actualizar · eliminar
evaluaciones	crear · leer · actualizar · eliminar
calificaciones	crear · leer · actualizar · eliminar
resultado_academico	generar · leer · eliminar
resultado_plan	leer · actualizar · eliminar
certificados	emitir · leer · actualizar · eliminar

4.3 JWT

Campo	Detalle
Ubicación del token	Header HTTP (JWT_TOKEN_LOCATION = ["headers"])
Expiración access token	15 minutos
Expiración refresh token	7 días
Clave de firma	JWT_SECRET_KEY (obligatoria por variable de entorno)

4.4 Redis

Redis se utiliza como almacén de sesiones y caché de permisos para AuthCommon, compartido entre los tres microservicios del sistema. El tiempo de vida de la sesión se configura vía AUTH_COMMON_SESSION_TTL (900 segundos por defecto).

5. Formato estándar de respuesta y códigos HTTP

5.1 Envelope de respuesta

Todas las respuestas del microservicio siguen un formato JSON estándar, generado por `utils/response.py` (`success_response` / `error_response`), utilizado de manera uniforme por los 8 controllers.

Respuesta exitosa

```
{
  "status": "success",
  "data": { ... } | [ ... ],
  "message": "Descripción de la operación realizada.",
  "total": 12 // solo presente en listados
}
```

Respuesta de error

```
{
  "status": "error",
  "message": "Descripción del error.",
  "errors": { "campo": ["mensaje de validación"] } // solo si aplica
}
```

5.2 Códigos HTTP utilizados

Campo	Tipo	Nulo	Clave	Descripción
200 OK	Éxito	—	—	Operaciones GET, PUT y DELETE exitosas.
201 Created	Éxito	—	—	Alta exitosa (POST).
400 Bad Request	Error	—	—	Error de validación de schema (<code>ValidationError</code>) o violación de una regla de negocio (<code>BusinessError</code> , código por defecto).
403 Forbidden	Error	—	—	El solicitante no cumple los requisitos de negocio para la acción (ej: legajo no cumple correlativas).
404 Not Found	Error	—	—	Recurso solicitado, o una entidad relacionada, no existe.
500 Internal Server Error	Error	—	—	Error de integridad de base de datos o excepción no controlada.

5.3 Manejo de errores

Todos los controllers siguen el mismo patrón de manejo de excepciones, envolviendo la llamada al service en un bloque `try/except`:

- `ValidationError` (Marshmallow): se traduce a 400, incluyendo el detalle de campos inválidos en `errors`.
- `BusinessError` (excepción propia, definida en `exceptions.py`): incluye su propio mensaje y `status_code`, definidos en el punto exacto del service donde se viola la regla de negocio. Esto permite que un mismo tipo de excepción represente 400, 403, 404 o 500 según el caso.

Definición de la excepción de negocio:

```
class BusinessError(Exception):  
    def __init__(self, message, status_code=400):  
        self.message = message  
        self.status_code = status_code
```

6. Modelo de datos

6.1 Diagrama entidad-relación

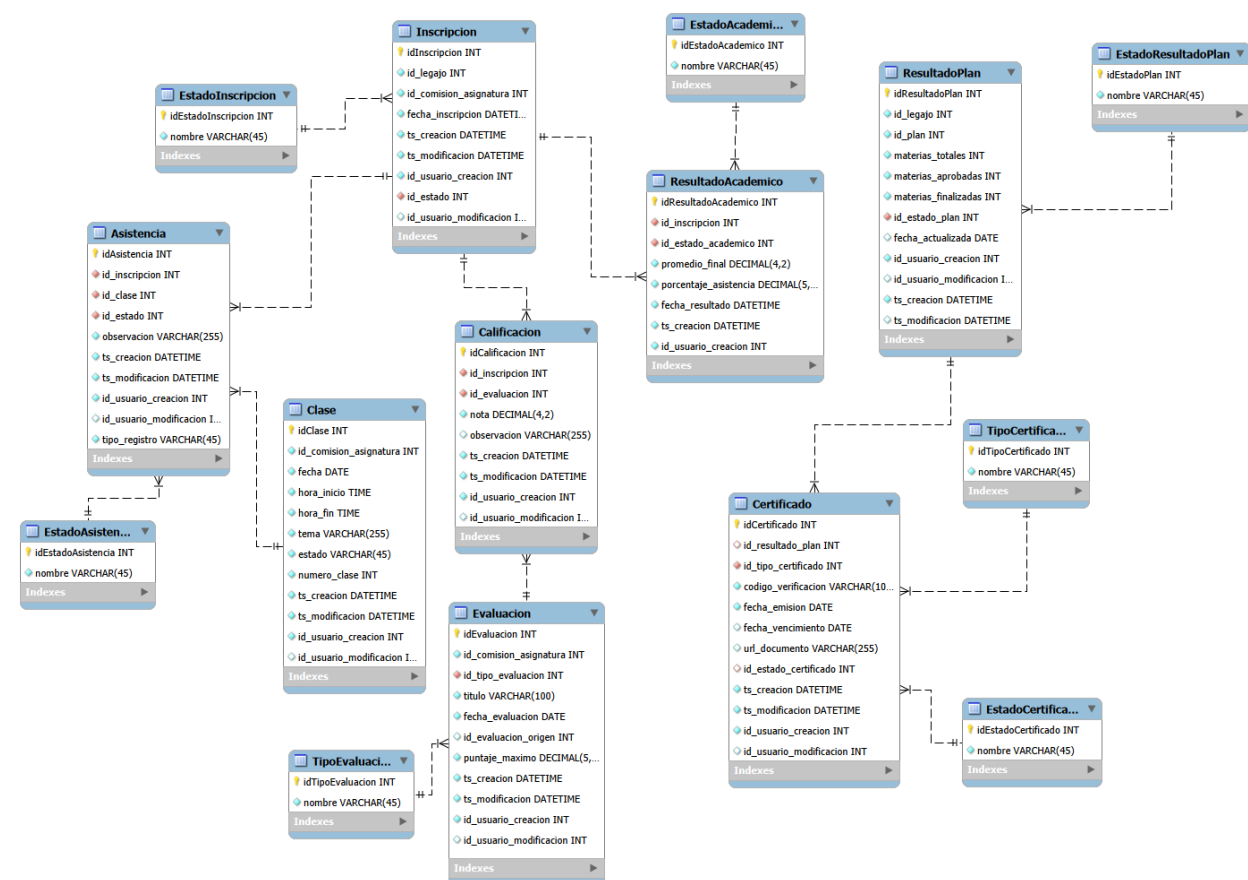


Figura 2. Modelo entidad-relación del Microservicio 2 — Inscripciones.

6.2 Referencias externas (a otros microservicios)

Varios campos no son claves foráneas de base de datos, sino referencias a entidades gestionadas por otro microservicio. Se validan en tiempo real vía HTTP contra el Microservicio 1 (Planes) y no poseen integridad referencial a nivel de base de datos local:

Campo	Detalle
id_legajo	Identificador del alumno/legajo, gestionado por el Microservicio 1.
id_comision_asignatura	Comisión-asignatura (curso) a la que pertenece la clase, evaluación o inscripción. Gestionada por el Microservicio 1.
id_plan	Plan de estudio del alumno, gestionado por el Microservicio 1.

6.3 Detalle de entidades

Inscripcion

Tabla: inscripciones

Campo	Tipo	Nulo	Clave	Descripción
id_inscripcion	Integer	No	PK	Autoincremental.
id_legajo	Integer	No	Ref. externa	Alumno inscripto (MS1).
id_comision_asignatura	Integer	No	Ref. externa	Comisión-asignatura (MS1).
id_estado	Integer	No	FK	→ estado_inscripcion.id_estado
fecha_inscripcion	DateTime	No	—	Fecha/hora de alta de la inscripción.
ts_creacion / ts_modificacion	DateTime	Sí (mod.)	—	Auditoría.
id_usuario_creacion / modificacion	Integer	Sí (mod.)	—	Auditoría (usuario autenticado).

Clase

Tabla: clase

Campo	Tipo	Nulo	Clave	Descripción
id_clase	Integer	No	PK	—
id_comision_asignatura	Integer	No	Ref. externa	Comisión-asignatura (MS1).
numero_clase	Integer	No	—	Único dentro de la comisión.
fecha / hora_inicio / hora_fin	Date / Time	No	—	hora_fin debe ser mayor que hora_inicio.
tema	String(255)	No	—	—
estado	Enum(EstadoClase)	No	—	PROGRAMADA · DICTADA · SUSPENDIDA · REPROGRAMADA. Default: PROGRAMADA.
auditoría	—	—	—	id_usuario/ts creación y modificación.

Asistencia

Tabla: asistencia

Campo	Tipo	Nulo	Clave	Descripción
id_asistencia	Integer	No	PK	—
id_inscripcion	Integer	No	FK	→ inscripciones.id_inscripcion
id_clase	Integer	No	FK	→ clase.id_clase
id_estado	Integer	No	FK	→ estado_asistencia.id_estado_asistencia
tipo_registro	Enum(TipoRegistro)	No	—	MANUAL · QR. Default: MANUAL.
observacion	String(255)	Sí	—	—
auditoría	—	—	—	id_usuario/ts creación y modificación.

Evaluacion

Tabla: evaluacion

Campo	Tipo	Nulo	Clave	Descripción
id_evaluacion	Integer	No	PK	—
id_comision_asignatura	Integer	No	Ref. externa	Comisión-asignatura (MS1).
id_tipo_evaluacion	Integer	No	FK	→ tipo_evaluacion.id_tipo_evaluacion
titulo	String(100)	No	—	—
fecha_evaluacion	Date	No	—	—
id_evaluacion_origen	Integer	Sí	FK (self)	Evaluación original si esta es un recuperatorio.
puntaje_maximo	Integer	No	—	Debe ser mayor a 0.
auditoría	—	—	—	id_usuario/ts creación y modificación.

Calificacion

Tabla: calificacion

Campo	Tipo	Nulo	Clave	Descripción
id_calificacion	Integer	No	PK	—
id_evaluacion	Integer	No	FK	→ evaluacion.id_evaluacion
id_inscripcion	Integer	No	FK	→ inscripciones.id_inscripcion
puntaje	Float	No	—	$0 \leq \text{puntaje} \leq \text{puntaje_maximo}$ de la evaluación.
observacion	String(255)	Sí	—	—
auditoría	—	—	—	id_usuario/ts creación y modificación.

ResultadoAcademico

Tabla: resultado_academico

Campo	Tipo	Nulo	Clave	Descripción
id_resultado_academico	Integer	No	PK	—
id_inscripcion	Integer	No	FK, unique	→ inscripciones.id_inscripcion (relación 1:1).
porcentaje_asistencia	Float	No	—	Calculado automáticamente.
promedio_final	Float	No	—	Calculado automáticamente.
id_estado_academico	Integer	No	FK	→ estado_academico.id_estado_academico
fecha_resultado	Date	No	—	—
id_usuario_creacion / ts_creacion	—	No	—	Auditoría. No posee campos de modificación (el resultado no se edita, solo se elimina en desarrollo).

ResultadoPlan

Tabla: resultado_plan

Campo	Tipo	Nulo	Clave	Descripción
id_resultado_plan	Integer	No	PK	—
id_legajo	Integer	No	Ref. externa	Alumno (MS1).
id_plan	Integer	No	Ref. externa	Plan de estudio (MS1).
materias_totales	Integer	No	—	Cantidad de comisiones-asignatura del plan.
materias_aprobadas	Integer	No	—	Cantidad de resultados académicos en estado Aprobado.
materias_finalizadas	Integer	No	—	Cantidad total de resultados académicos generados.
id_estado_resultado_plan	Integer	No	FK	→ estado_resultado_plan.id_estado_resultado_plan
fecha_actualizacion	Date	No	—	Se actualiza en cada recálculo.
auditoría	—	—	—	id_usuario/ts creación y modificación.

Certificado

Tabla: certificado

Campo	Tipo	Nulo	Clave	Descripción
id_certificado	Integer	No	PK	—
id_resultado_plan	Integer	No	FK, unique	→ resultado_plan.id_resultado_plan (relación 1:1).
id_tipo_certificado	Integer	No	FK	→ tipo_certificado.id_tipo_certificado
codigo_verificacion	String(50)	No	Unique	Generado automáticamente: CERT-XXXXXXX (8 hex de un UUID4, mayúsculas).

Campo	Tipo	Nulo	Clave	Descripción
fecha_emision	Date	No	—	Fecha de creación del certificado.
fecha_vencimiento	Date	Sí	—	null por defecto; editable vía PUT.
url_documento	String(255)	Sí	—	null por defecto; editable vía PUT.
id_estado_certificado	Integer	No	FK	→ estado_certificado.id_estado_certificado
auditoría	—	—	—	id_usuario/ts creación y modificación.

6.4 Catálogos y valores iniciales (seed)

Los siguientes catálogos se cargan automáticamente al iniciar la aplicación (seed/seed_data.py), únicamente si la tabla correspondiente está vacía, evitando duplicados en reinicios sucesivos:

Campo	Detalle
estado_inscripcion	Pendiente (1) · Aceptada (2) · Rechazada (3) · Cancelada (4) · Finalizada (5)
estado_asistencia	Presente (1) · Ausente (2) · Justificado (3) · Tarde (4)
estado_academico	Regular (1) · Aprobado (2) · Desaprobado (3) · Libre (4)
estado_resultado_plan	En curso (1) · Finalizado (2) · Incompleto (3) · Abandonado (4)
estado_certificado	Emitido (1) · Revocado (2)
tipo_evaluacion	Parcial (1) · Recuperatorio (2) · Final (3) · Trabajo Práctico (4)
tipo_certificado	Aprobación (1) · Participación (2)

Nota: el estado 2 ("Aceptada") de estado_inscripcion y el enum EstadoClase (definido en el propio modelo, no en base de datos) son referenciados por identificador o nombre fijo en varios puntos de la lógica de negocio (ver sección 8), por lo que no deben eliminarse ni renombrarse sin actualizar el código.

7. Comunicación entre microservicios

7.1 Cliente hacia el Microservicio 1 (Planes)

El módulo `clients/planes_cliente.py` centraliza toda la comunicación saliente hacia el Microservicio 1. Es utilizado por los services de Clase, Evaluación, Inscripción, ResultadoAcademico y ResultadoPlan para validar y obtener datos de legajos, comisiones-asignatura y planes de estudio.

Campo	Detalle
Base URL	Configurable vía <code>PLANES_SERVICE_URL</code> .
Método HTTP	GET (todas las funciones son de solo lectura).
Timeout	10 segundos por petición.
Propagación de headers	Se reenvían los headers del request original (incluye el JWT del usuario autenticado).
Manejo de errores	Ante cualquier error de red o HTTP, se registra en log y la función devuelve <code>None</code> — el service que la invoca es responsable de traducir ese <code>None</code> en un <code>BusinessError 404</code> .

Endpoints consumidos del Microservicio 1

Método	Path	Permiso requerido	Descripción
GET	<code>/comisiones-asignaturas/GetDetalleFromLegajoID</code>	—	Lista de comisiones-asignatura disponibles (opcionalmente filtradas por legajo).
GET	<code>/legajos/GetPersonaFromLegajoID</code>	—	Datos del legajo/alumno por ID.

A partir de la respuesta de comisiones-asignatura, el cliente deriva localmente (sin llamadas HTTP adicionales) los datos de `plan_asignaturas` y plan asociados a cada comisión, ya que vienen anidados en la misma respuesta.

Datos consumidos de las reglas académicas del plan

Al generar resultados académicos, el service consulta las reglas del plan de asignatura asociado a la comisión, esperando (como mínimo) los siguientes campos en la respuesta del Microservicio 1:

Campo	Detalle
<code>presentismo_porc</code>	Porcentaje mínimo de asistencia requerido para no quedar Libre.
<code>final_aprobacion</code>	Promedio mínimo para el estado Aprobado.
<code>regularizacion_prom</code>	Promedio mínimo para el estado Regular.
<code>cupos_maximo</code>	Cupo máximo de la comisión-asignatura (usado al crear/modificar inscripciones).

7.2 Comunicación hacia el Microservicio Auth

Campo	Detalle
Registro de permisos	Al iniciar la app, POST {AUTH_SERVICE_URL}/auth/acciones con el contenido de acciones.yml.
Validación de sesión/token	Delegada a la librería AuthCommon en cada request entrante (ver sección 4).

8. Catálogo de endpoints

Esta sección detalla, módulo por módulo, cada endpoint expuesto por el microservicio: método HTTP, path, permiso requerido, estructura del request, estructura del response, códigos de error posibles y las reglas de negocio aplicadas antes de ejecutar la operación.

Todas las rutas de este microservicio requieren autenticación (JWT) salvo /status. Todos los paths listados a continuación son relativos a la URL base del microservicio.

8.1 Inscripciones — /inscripciones

Método	Path	Permiso requerido	Descripción
POST	/inscripciones/	inscripcion.inscripciones.crear	Registrar una inscripción.
GET	/inscripciones/	inscripcion.inscripciones.leer	Listar inscripciones (filtros).
GET	/inscripciones/{id}	inscripcion.inscripciones.leer	Obtener una inscripción.
PUT	/inscripciones/{id}	inscripcion.inscripciones.actualizar	Modificar estado o comisión.
DELETE	/inscripciones/{id}	inscripcion.inscripciones.eliminar	Eliminar una inscripción.

POST /inscripciones/ — Registrar inscripción

Permiso requerido: inscripcion.inscripciones.crear

Request body:

```
{
  "id_legajo": 1024,
  "id_comision_asignatura": 55
}
```

Response 201:

```
{
  "status": "success",
  "message": "Solicitud de inscripción enviada correctamente",
  "data": {
    "id_inscripcion": 12,
    "id_legajo": 1024,
    "id_comision_asignatura": 55,
    "id_estado": 1,
    "estado": { "id_estado": 1, "nombre": "Pendiente" },
    "fecha_inscripcion": "2026-07-30T10:00:00"
  }
}
```

Errores posibles: 400 validación / datos duplicados / sin cupo · 403 no cumple requisitos de correlatividad · 404 legajo o comisión inexistente

Reglas de negocio:

- Valida que el legajo exista, consultando al Microservicio 1.
- Valida que la comisión-asignatura exista (consulta general al Microservicio 1).
- Valida que el legajo cumpla los requisitos académicos (correlativas) para esa comisión — consulta específica filtrada por legajo al Microservicio 1; si no está en el listado devuelto, se rechaza con 403.

- Valida que la comisión tenga cupo disponible: cuenta inscripciones en estado Aceptada o Finalizada y las compara contra cupo_maximo.
- No permite doble inscripción del mismo legajo en la misma comisión.
- El estado inicial de toda inscripción nueva es siempre "Pendiente".

GET /inscripciones/ — *Listar inscripciones*

Permiso requerido: inscripcion.inscripciones.leer

Parámetros de query: id_estado (int) · id_legajo (int) · id_comision_asignatura (int) — todos opcionales, combinables.

Response 200:

```
{
  "status": "success",
  "data": [ { "id_inscripcion": 12, "...": "..." } ],
  "message": "Listado de inscripciones.",
  "total": 1
}
```

GET /inscripciones/{id_inscripcion} — *Obtener inscripción por ID*

Permiso requerido: inscripcion.inscripciones.leer

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Inscripción encontrada." }
```

Errores posibles: 404 si no existe.

PUT /inscripciones/{id_inscripcion} — *Modificar inscripción*

Permiso requerido: inscripcion.inscripciones.actualizar

Request body:

```
{
  "id_estado": 2,
  "id_comision_asignatura": 60
}
// ambos campos son opcionales
```

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Inscripción 12 actualizada." }
```

Errores posibles: 400 estado o comisión inválidos / sin cupo · 404 no encontrada / comisión inexistente

Reglas de negocio:

- Si se cambia id_estado y el nuevo estado es "Aceptada" (y antes no lo estaba), se dispara automáticamente la actualización del ResultadoPlan del alumno para ese plan de estudio.
- Si se cambia id_comision_asignatura, se revalida la existencia de la nueva comisión y su cupo disponible.
- Limitación conocida: al cambiar de comisión no se libera el cupo que el alumno ocupaba en la comisión anterior (pendiente de resolver cuando el Microservicio 1 exponga API de cupos).

DELETE /inscripciones/{id_inscripcion} — *Eliminar inscripción*

Permiso requerido: inscripcion.inscripciones.eliminar

Response 200:

```
{ "status": "success", "message": "Inscripción 12 eliminada." }
```

Errores posibles: 400 si posee asistencias o calificaciones asociadas · 404 no encontrada

Reglas de negocio:

- No permite eliminar una inscripción que ya tenga asistencias o calificaciones registradas.

8.2 Clases — /clases

Método	Path	Permiso requerido	Descripción
POST	/clases/	inscripcion.clases.crear	Crear una clase.
GET	/clases/	inscripcion.clases.leer	Listar clases (filtros).
GET	/clases/{id}	inscripcion.clases.leer	Obtener una clase.
PUT	/clases/{id}	inscripcion.clases.actualizar	Modificar una clase.
DELETE	/clases/{id}	inscripcion.clases.eliminar	Eliminar una clase.

POST /clases/ — *Crear clase*

Permiso requerido: inscripcion.clases.crear

Request body:

```
{
  "id_comision_asignatura": 55,
  "numero_clase": 1,
  "fecha": "2026-08-03",
  "hora_inicio": "18:00:00",
  "hora_fin": "20:00:00",
  "tema": "Introducción a la materia"
}
```

Response 201:

```
{ "status": "success", "message": "Clase creada correctamente.", "data": { "id_clase": 5,
"estado": "PROGRAMADA", "...": "..." } }
```

Errores posibles: 400 validación / número de clase duplicado / horario inválido · 404 comisión inexistente

Reglas de negocio:

- Valida que la comisión-asignatura exista, consultando al Microservicio 1.
- No permite repetir numero_clase dentro de la misma comisión.
- hora_fin debe ser mayor que hora_inicio.
- Toda clase nueva se crea con estado PROGRAMADA.

GET /clases/ — *Listar clases*

Permiso requerido: inscripcion.clases.leer

Parámetros de query: id_comision_asignatura (int) · estado (string, ej: DICTADA — se normaliza a mayúsculas y se mapea al Enum EstadoClase).

Response 200:

```
{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de clases.",
"total": 1 }
```

GET /clases/{id_clase} — *Obtener clase por ID*

Permiso requerido: inscripcion.clases.leer

Response 200:

```
{ "status": "success", "data": { "...": "..."}, "message": "Clase encontrada." }
```

Errores posibles: 404 si no existe.

PUT /clases/{id_clase} — *Modificar clase*

Permiso requerido: inscripcion.clases.actualizar

Request body:

```
{
  "id_comision_asignatura": 55, // requerido por el schema, no puede diferir del actual
  "numero_clase": 2,
  "fecha": "2026-08-05",
  "hora_inicio": "18:00:00",
  "hora_fin": "20:00:00",
  "tema": "Nuevo tema",
  "estado": "DICTADA"
}
```

Response 200:

```
{ "status": "success", "data": { "...": "..."}, "message": "Clase 5 actualizada." }
```

Errores posibles: 400 número de clase duplicado / horario inválido / intento de cambiar comisión · 404 no encontrada

Reglas de negocio:

- No es posible modificar la comisión-asignatura de una clase existente (aunque el schema exige el campo, el service rechaza cualquier valor distinto al actual).
- Si se cambia numero_clase, se revalida que no exista otra clase con ese número en la misma comisión.
- Se revalida hora_fin > hora_inicio tras aplicar los cambios.
- estado se recibe como string y se mapea al Enum EstadoClase (PROGRAMADA · DICTADA · SUSPENDIDA · REPROGRAMADA).

DELETE /clases/{id_clase} — *Eliminar clase*

Permiso requerido: inscripcion.clases.eliminar

Response 200:

```
{ "status": "success", "message": "Clase 5 eliminada." }
```

Errores posibles: 400 si posee asistencias registradas · 404 no encontrada

Reglas de negocio:

- No permite eliminar una clase que ya tenga asistencias registradas.

8.3 Asistencias — /asistencias

El registro de asistencia se realiza en lote: un único request agrupa todas las asistencias de una misma clase (una por inscripción).

Método	Path	Permiso requerido	Descripción
POST	/asistencias/	inscripcion.asistencias.crear	Registrar asistencias de una clase (lote).
GET	/asistencias/	inscripcion.asistencias.leer	Listar asistencias (filtros).
GET	/asistencias/{id}	inscripcion.asistencias.leer	Obtener una asistencia.
PUT	/asistencias/{id}	inscripcion.asistencias.actualizar	Modificar una asistencia.
DELETE	/asistencias/{id}	inscripcion.asistencias.eliminar	Eliminar una asistencia (solo desarrollo).

POST /asistencias/ — Registrar asistencias de una clase

Permiso requerido: inscripcion.asistencias.crear

Request body:

```
{
  "id_clase": 5,
  "asistencias": [
    { "id_inscripcion": 12, "id_estado": 1, "observacion": null },
    { "id_inscripcion": 13, "id_estado": 2 }
  ]
}
```

Response 201:

```
{ "status": "success", "message": "Asistencias registradas correctamente.", "total": 2,
  "data": [ { "...": "..." } ] }
```

Errores posibles: 400 clase no dictada / duplicados / lista vacía / estado inválido / inscripción no pertenece a la comisión · 404 clase o inscripción inexistente

Reglas de negocio:

- La clase debe existir y encontrarse en estado DICTADA — no se puede tomar asistencia de una clase Programada, Suspendida o Reprogramada.
- Cada inscripción debe existir y pertenecer a la misma comisión-asignatura que la clase.
- El id_estado enviado debe corresponder a un estado de asistencia existente.
- No se permite registrar dos veces asistencia para la misma inscripción en la misma clase (ni dentro del mismo request, ni contra registros ya existentes).
- El tipo_registro se fija automáticamente en MANUAL para altas por este endpoint.

GET /asistencias/ — Listar asistencias

Permiso requerido: inscripcion.asistencias.leer

Parámetros de query: id_clase (int) · id_inscripcion (int) — opcionales, combinables.

Response 200:

```
{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de asistencias.",
  "total": 2 }
```

GET /asistencias/{id_asistencia} — Obtener asistencia por ID

Permiso requerido: inscripcion.asistencias.leer

Response 200:

```
{ "status": "success", "data": { "...": "...", "message": "Asistencia encontrada." }
```

Errores posibles: 404 si no existe.

PUT /asistencias/{id_asistencia} — Modificar asistencia

Permiso requerido: inscripcion.asistencias.actualizar

Request body:

```
{ "id_estado": 3, "observacion": "Llegó 20 minutos tarde" }  
// ambos campos opcionales
```

Response 200:

```
{ "status": "success", "data": { "...": "...", "message": "Asistencia 45 actualizada." }
```

Errores posibles: 400 clase no dictada / estado inválido · 404 no encontrada / clase inexistente

Reglas de negocio:

- Solo pueden modificarse asistencias de clases en estado DICTADA.
- Si no hay cambios reales en los valores enviados, no se ejecuta UPDATE ni se actualiza auditoría (optimización de escritura).

DELETE /asistencias/{id_asistencia} — Eliminar asistencia

Permiso requerido: inscripcion.asistencias.eliminar

Response 200:

```
{ "status": "success", "message": "Asistencia 45 eliminada." }
```

Errores posibles: 404 si no existe.

Reglas de negocio:

- Habilitado únicamente para uso en entornos de desarrollo, según indica el comentario en la ruta.

8.4 Evaluaciones — /evaluaciones

Método	Path	Permiso requerido	Descripción
POST	/evaluaciones/	inscripcion.evaluaciones.crear	Crear una evaluación.
GET	/evaluaciones/	inscripcion.evaluaciones.leer	Listar evaluaciones (filtros).
GET	/evaluaciones/{id}	inscripcion.evaluaciones.leer	Obtener una evaluación.
PUT	/evaluaciones/{id}	inscripcion.evaluaciones.actualizar	Modificar una evaluación.
DELETE	/evaluaciones/{id}	inscripcion.evaluaciones.eliminar	Eliminar una evaluación.

POST /evaluaciones/ — Crear evaluación

Permiso requerido: inscripcion.evaluaciones.crear

Request body:

```
{
```

```

    "id_comision_asignatura": 55,
    "id_tipo_evaluacion": 1,
    "titulo": "Primer parcial",
    "fecha_evaluacion": "2026-09-10",
    "puntaje_maximo": 100,
    "id_evaluacion_origen": null
  }

```

Response 201:

```

{ "status": "success", "message": "Evaluación creada correctamente.", "data": { "...":
"..."} }

```

Errores posibles: 400 puntaje_maximo ≤ 0 · 404 comisión / tipo de evaluación / evaluación origen inexistente

Reglas de negocio:

- Valida que la comisión-asignatura exista, consultando al Microservicio 1.
- Valida que el tipo de evaluación exista en el catálogo local.
- puntaje_maximo debe ser mayor a cero.
- Si se indica id_evaluacion_origen (recuperatorio), valida que la evaluación origen exista.

GET /evaluaciones/ — Listar evaluaciones

Permiso requerido: inscripcion.evaluaciones.leer

Parámetros de query: id_comision_asignatura (int) · id_tipo_evaluacion (int) — opcionales, combinables.

Response 200:

```

{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de evaluaciones.",
"total": 1 }

```

GET /evaluaciones/{id_evaluacion} — Obtener evaluación por ID

Permiso requerido: inscripcion.evaluaciones.leer

Response 200:

```

{ "status": "success", "data": { "...": "..." }, "message": "Evaluación encontrada." }

```

Errores posibles: 404 si no existe.

PUT /evaluaciones/{id_evaluacion} — Modificar evaluación

Permiso requerido: inscripcion.evaluaciones.actualizar

Request body:

```

{ "id_comision_asignatura": 55, "titulo": "Primer parcial (recuperado)", "puntaje_maximo":
100 }
// solo id_comision_asignatura es requerido por el schema; el resto es opcional

```

Response 200:

```

{ "status": "success", "data": { "...": "..." }, "message": "Evaluación 8 actualizada." }

```

Errores posibles: 400 puntaje inválido / autoreferencia como origen · 404 comisión / tipo / evaluación origen inexistente

Reglas de negocio:

- Si se cambia id_comision_asignatura, se revalida su existencia contra el Microservicio 1.

- Si se cambia `id_tipo_evaluacion`, se revalida su existencia en el catálogo.
- Si se cambia `puntaje_maximo`, debe seguir siendo mayor a cero.
- Si se cambia `id_evaluacion_origen`: no puede ser la propia evaluación (autoreferencia) y debe existir.
- Solo se actualiza auditoría si hubo al menos un cambio real.

DELETE `/evaluaciones/{id_evaluacion}` — *Eliminar evaluación*

Permiso requerido: `inscripcion.evaluaciones.eliminar`

Response 200:

```
{ "status": "success", "message": "Evaluación 8 eliminada." }
```

Errores posibles: 400 si posee calificaciones registradas · 404 no encontrada

Reglas de negocio:

- No permite eliminar una evaluación que ya tenga calificaciones registradas.

8.5 Calificaciones — /calificaciones

Al igual que asistencias, el registro es en lote: un request agrupa todas las calificaciones de una misma evaluación.

Método	Path	Permiso requerido	Descripción
POST	/calificaciones/	inscripcion.calificaciones.crear	Registrar calificaciones de una evaluación (lote).
GET	/calificaciones/	inscripcion.calificaciones.leer	Listar calificaciones (filtros).
GET	/calificaciones/{id}	inscripcion.calificaciones.leer	Obtener una calificación.
PUT	/calificaciones/{id}	inscripcion.calificaciones.actualizar	Modificar una calificación.
DELETE	/calificaciones/{id}	inscripcion.calificaciones.eliminar	Eliminar una calificación.

POST /calificaciones/ — Registrar calificaciones de una evaluación

Permiso requerido: inscripcion.calificaciones.crear

Request body:

```
{
  "id_evaluacion": 8,
  "calificaciones": [
    { "id_inscripcion": 12, "puntaje": 85, "observacion": null },
    { "id_inscripcion": 13, "puntaje": 60 }
  ]
}
```

Response 201:

```
{ "status": "success", "message": "Calificaciones registradas correctamente.", "total": 2,
  "data": [ { "...": "..." } ] }
```

Errores posibles: 400 evaluación no existe / duplicados / lista vacía / puntaje fuera de rango / inscripción no pertenece a la comisión · 404 inscripción inexistente

Reglas de negocio:

- La evaluación debe existir.
- Cada inscripción debe existir y pertenecer a la misma comisión-asignatura que la evaluación.
- No se permite calificar dos veces la misma inscripción para la misma evaluación (ni dentro del mismo request, ni contra registros existentes).
- El puntaje no puede ser negativo ni superar el puntaje_maximo definido en la evaluación.

GET /calificaciones/ — Listar calificaciones

Permiso requerido: inscripcion.calificaciones.leer

Parámetros de query: id_evaluacion (int) · id_inscripcion (int) — opcionales, combinables.

Response 200:

```
{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de
calificaciones.", "total": 2 }
```

GET /calificaciones/{id_calificacion} — Obtener calificación por ID

Permiso requerido: inscripcion.calificaciones.leer

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Calificación encontrada." }
```

Errores posibles: 404 si no existe.

PUT /calificaciones/{id_calificacion} — *Modificar calificación*

Permiso requerido: inscripcion.calificaciones.actualizar

Request body:

```
{ "puntaje": 90, "observacion": "Recalificado tras revisión" }  
// ambos campos opcionales
```

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Calificación 30 actualizada." }
```

Errores posibles: 400 puntaje negativo o fuera de rango · 404 no encontrada / evaluación inexistente

Reglas de negocio:

- Si se cambia puntaje, se revalida contra el rango [0, puntaje_maximo] de la evaluación asociada.
- Solo se actualiza auditoría si hubo un cambio real.

DELETE /calificaciones/{id_calificacion} — *Eliminar calificación*

Permiso requerido: inscripcion.calificaciones.eliminar

Response 200:

```
{ "status": "success", "message": "Calificación 30 eliminada." }
```

Errores posibles: 404 si no existe.

8.6 Resultados académicos — /resultados-academicos

Este recurso no se crea individualmente: se genera de forma masiva para todos los alumnos aceptados de una comisión-asignatura que ya finalizó su dictado. No posee endpoint de modificación.

Método	Path	Permiso requerido	Descripción
POST	/resultados-academicos/	inscripcion.resultado_academico.generar	Generar resultados académicos de una comisión.
GET	/resultados-academicos/	inscripcion.resultado_academico.leer	Listar todos los resultados académicos.
GET	/resultados-academicos/{id}	inscripcion.resultado_academico.leer	Obtener un resultado académico.
DELETE	/resultados-academicos/{id}	inscripcion.resultado_academico.eliminar	Eliminar un resultado (solo desarrollo).

POST /resultados-academicos/ — *Generar resultados académicos de una comisión*

Permiso requerido: inscripcion.resultado_academico.generar

Request body:

```
{ "id_comision_asignatura": 55 }
```

Response 201:

```
{ "status": "success", "message": "Resultados académicos generados correctamente.",  
  "total": 2, "data": [ { "id_resultado_academico": 1, "porcentaje_asistencia": 87.5,
```

```
"promedio_final": 78.0, "id_estado_academico": 2, "estado": { "nombre": "Aprobado" },
"...": "..." } ] }
```

Errores posibles: 400 comisión no finalizada / sin alumnos aceptados / todos ya poseen resultado / inscripción no aceptada

Reglas de negocio:

- La comisión debe estar finalizada: no puede tener clases en estado PROGRAMADA o REPROGRAMADA.
- Se procesan únicamente las inscripciones en estado Aceptada de esa comisión.
- Operación idempotente por alumno: si una inscripción ya posee resultado académico, se omite sin error (continúa con las siguientes).
- % de asistencia = (asistencias marcadas "Presente") / (clases en estado DICTADA de la comisión). Si aún no hubo clases dictadas, el porcentaje es 0.
- Promedio final = promedio simple de todas las calificaciones de la inscripción. Si no hay calificaciones, el promedio es 0.
- Determinación del estado académico, en este orden de prioridad, usando las reglas del plan (consultadas al Microservicio 1): si % asistencia < presentismo_porc → Libre; si no, si promedio ≥ final_aprobacion → Aprobado; si no, si promedio ≥ regularizacion_prom → Regular; caso contrario → Desaprobado.
- Efecto en cascada: al generarse el resultado, la inscripción pasa automáticamente a estado "Finalizada" y se dispara la actualización del ResultadoPlan del alumno.

GET /resultados-academicos/ — *Listar resultados académicos*

Permiso requerido: inscripcion.resultado_academico.leer

Response 200:

```
{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de resultados académicos.", "total": 1 }
```

GET /resultados-academicos/{id_resultado_academico} — *Obtener resultado académico por ID*

Permiso requerido: inscripcion.resultado_academico.leer

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Resultado académico encontrado." }
```

Errores posibles: 404 si no existe.

DELETE /resultados-academicos/{id_resultado_academico} — *Eliminar resultado académico*

Permiso requerido: inscripcion.resultado_academico.eliminar

Response 200:

```
{ "status": "success", "message": "Resultado académico 1 eliminado." }
```

Errores posibles: 404 si no existe.

Reglas de negocio:

- Habilitado únicamente para uso en entornos de desarrollo, según indica el comentario en el service.

8.7 Resultados de plan — /resultados-planes

Representa el avance del alumno dentro de su plan de estudio. No expone un endpoint de alta pública: se crea y actualiza automáticamente (upsert) desde otros procesos de negocio — al aceptar una inscripción y al generar un resultado académico (ver secciones 8.1 y 8.6).

Método	Path	Permiso requerido	Descripción
GET	/resultados-planes/	inscripcion.resultado_plan.leer	Listar resultados de plan.
GET	/resultados-planes/{id}	inscripcion.resultado_plan.leer	Obtener un resultado de plan.
PUT	/resultados-planes/{id}	inscripcion.resultado_plan.actualizar	Cambiar el estado del plan (uso administrativo).
DELETE	/resultados-planes/{id}	inscripcion.resultado_plan.eliminar	Eliminar un resultado de plan.

Nota técnica: el service resultado_plan_service.py define una función de alta/actualización (actualizar_resultado_plan) y su controller (agregar_resultado_plan), pero esta acción no está registrada como ruta pública en resultado_plan_routes.py. Es, en la práctica, una función interna reutilizable, no un endpoint expuesto al cliente.

GET /resultados-planes/ — Listar resultados de plan

Permiso requerido: inscripcion.resultado_plan.leer

Response 200:

```
{ "status": "success", "data": [ { "id_resultado_plan": 3, "materias_totales": 8,
  "materias_aprobadas": 5, "materias_finalizadas": 6, "estado": { "nombre": "En curso" },
  "...": "..." } ], "message": "Listado de resultados de planes.", "total": 1 }
```

GET /resultados-planes/{id_resultado_plan} — Obtener resultado de plan por ID

Permiso requerido: inscripcion.resultado_plan.leer

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Resultado del plan
encontrado." }
```

Errores posibles: 404 si no existe.

PUT /resultados-planes/{id_resultado_plan} — Cambiar estado del resultado de plan

Permiso requerido: inscripcion.resultado_plan.actualizar

Request body:

```
{ "id_estado_resultado_plan": 4 }
// requerido — pensado para uso administrativo, ej. marcar "Abandonado"
```

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Resultado del plan 3
actualizado." }
```

Errores posibles: 404 no encontrado / estado inexistente

Reglas de negocio:

- Solo aplica cambios si el estado enviado es distinto al actual.

- Efecto en cascada crítico: si el nuevo estado es "Abandonado", se cancelan automáticamente TODAS las inscripciones activas del alumno correspondientes a ese plan de estudio (pasan a estado "Cancelada").

DELETE /resultados-planes/{id_resultado_plan} — *Eliminar resultado de plan*

Permiso requerido: inscripcion.resultado_plan.eliminar

Response 200:

```
{ "status": "success", "message": "Resultado del plan 3 eliminado." }
```

Errores posibles: 404 si no existe.

8.8 Certificados — /certificados

Método	Path	Permiso requerido	Descripción
POST	/certificados/	inscripcion.certificados.emitir	Emitir un certificado.
GET	/certificados/	inscripcion.certificados.leer	Listar certificados.
GET	/certificados/{id}	inscripcion.certificados.leer	Obtener un certificado.
PUT	/certificados/{id}	inscripcion.certificados.actualizar	Modificar un certificado.
DELETE	/certificados/{id}	inscripcion.certificados.eliminar	Eliminar un certificado.

POST /certificados/ — Emitir certificado

Permiso requerido: inscripcion.certificados.emitir

Request body:

```
{ "id_resultado_plan": 3 }
```

Response 201:

```
{
  "status": "success",
  "message": "Certificado emitido correctamente.",
  "data": {
    "id_certificado": 7,
    "codigo_verificacion": "CERT-A1B2C3D4",
    "fecha_emision": "2026-07-30",
    "fecha_vencimiento": null,
    "tipo": { "nombre": "Aprobación" },
    "estado": { "nombre": "Emitido" },
    "...": "..."
  }
}
```

Errores posibles: 400 plan abandonado / plan no finalizado / certificado ya existente · 404 resultado de plan inexistente / tipo o estado de certificado inexistente

Reglas de negocio:

- El resultado de plan indicado no puede estar en estado "Abandonado".
- El plan debe estar completo: materias_finalizadas debe ser igual a materias_totales.
- No puede existir más de un certificado por resultado de plan (relación 1:1).
- El tipo de certificado se determina automáticamente según el estado del plan: "Finalizado" → certificado de Aprobación; "Incompleto" → certificado de Participación.
- El código de verificación se autogenera con el formato CERT-XXXXXXX (8 caracteres hexadecimales de un UUID4, en mayúsculas), garantizando unicidad.
- El estado inicial del certificado es siempre "Emitido", con fecha de emisión igual al día de la operación y sin fecha de vencimiento.

GET /certificados/ — Listar certificados

Permiso requerido: inscripcion.certificados.leer

Response 200:

```
{ "status": "success", "data": [ { "...": "..." } ], "message": "Listado de certificados.", "total": 1 }
```

GET /certificados/{id_certificado} — *Obtener certificado por ID*

Permiso requerido: inscripcion.certificados.leer

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Certificado encontrado." }
```

Errores posibles: 404 si no existe.

PUT /certificados/{id_certificado} — *Modificar certificado*

Permiso requerido: inscripcion.certificados.actualizar

Request body:

```
{ "id_estado_certificado": 2, "url_documento": "https://.../certificado.pdf",  
  "fecha_vencimiento": "2027-07-30" }  
// todos los campos opcionales
```

Response 200:

```
{ "status": "success", "data": { "...": "..." }, "message": "Certificado 7 actualizado." }
```

Errores posibles: 404 no encontrado / estado de certificado inexistente

Reglas de negocio:

- Permite cambiar el estado (ej. a "Revocado"), asociar la URL del documento generado y/o definir una fecha de vencimiento.

DELETE /certificados/{id_certificado} — *Eliminar certificado*

Permiso requerido: inscripcion.certificados.eliminar

Response 200:

```
{ "status": "success", "message": "Certificado 7 eliminado." }
```

Errores posibles: 404 si no existe.

9. Reglas de negocio transversales

Patrones aplicados de manera consistente en todos los módulos del microservicio:

- Auditoría uniforme: toda entidad (salvo catálogos) registra `id_usuario_creacion`, `id_usuario_modificacion`, `ts_creacion` y `ts_modificacion`, tomando el usuario autenticado desde `g.id_usuario` (inyectado por `AuthCommon`).
- Actualizaciones inteligentes: los servicios de modificación (asistencia, calificación, evaluación) comparan valor actual vs. valor recibido y solo ejecutan `UPDATE` si hubo un cambio real, evitando escrituras y actualizaciones de auditoría innecesarias.
- Protección de integridad referencial a nivel de negocio: no se permite eliminar clases con asistencias, evaluaciones con calificaciones, ni inscripciones con asistencias o calificaciones asociadas.
- Manejo uniforme de errores en todos los services: `IntegrityError` de base de datos → `BusinessError 500`; `BusinessError` se relanza tal cual; cualquier otra excepción → `BusinessError 500` genérico. Todas las operaciones de escritura hacen `rollback` ante error.
- Alta en lote para registros dependientes de una clase o evaluación (asistencias y calificaciones), con verificación de duplicados tanto dentro del request como contra la base de datos.
- Efectos en cascada entre módulos: aceptar una inscripción, generar un resultado académico, y abandonar un plan disparan actualizaciones automáticas en otras entidades (`ResultadoPlan` e `Inscripcion`), sin intervención manual del cliente.

10. Limitaciones conocidas y puntos pendientes

Relevadas directamente del código fuente (comentarios y comportamiento observado):

- Al cambiar la comisión-asignatura de una inscripción existente, no se libera el cupo que el alumno ocupaba en la comisión anterior — pendiente hasta que el Microservicio 1 exponga una API de gestión de cupos (comentario explícito en `inscripcion_service.py`).
- Los endpoints `DELETE` de Asistencia y de Resultado Académico están marcados en el propio código como habilitados "solo para desarrollo" — se recomienda restringir o remover estos permisos en un entorno de producción.
- El schema `ModificarClaseSchema` exige `id_comision_asignatura` como campo obligatorio en el `PUT`, aunque la regla de negocio impide modificarlo — cualquier valor distinto al actual es rechazado por el service. Se recomienda hacerlo opcional en el schema, o documentar explícitamente que debe enviarse igual al valor vigente.
- El endpoint de alta/actualización de Resultado de Plan (`agregar_resultado_plan / actualizar_resultado_plan`) existe a nivel de controller y service, pero no está registrado como ruta pública — queda como función interna. Si se desea exponerlo, debe agregarse su `Blueprint` en `resultado_plan_routes.py`.
- La validación de existencia de comisiones-asignatura y legajos depende enteramente de la disponibilidad del Microservicio 1 en tiempo real; no existe caché local ni tolerancia a fallos ante una caída de ese servicio (las funciones del cliente devuelven `None` y el flujo se corta con un error 404).